Practice Set: Method Overriding, Base Class Pointers, and Runtime Polymorphism

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define **Method Overriding** in C++ and distinguish it from **Function Overloading**. Highlight key differences including scope, function signature rules, binding types, and inheritance requirements.

Question 2: Explain why method overriding requires an identical function signature across base and derived classes. What happens if the derived class method parameter list differs slightly from the base class method?

Question 3: Explain the **Static Binding Gotcha** in C++ when invoking an overridden function through a base class pointer without using the virtual keyword.

Question 4: Explain the architectural need for **Base Class Pointers** using the **Universal Remote Control** analogy. How do base class pointers provide runtime flexibility in software design?

Question 5: What is a **Virtual Function** in C++? Explain how the virtual keyword alters method call resolution from **Static/Early Binding** to **Dynamic/Late Binding**.

Question 6: Detail the underlying internal mechanics of **vtable (Virtual Table)** and **vptr (Virtual Pointer)**. Explain how the compiler uses these structures to achieve dynamic dispatch at runtime.

Question 7: Step through the exact four-step memory execution sequence that takes place in RAM when a program executes ptr->show() using a base class pointer targeting a derived object.

Question 8: Explain the concept of **Object Slicing** in C++. What happens to dynamic polymorphism when a derived class object is assigned directly to a base class value object (e.g., Parent p = childObj;)?

Question 9: Explain the importance of the override specifier (C++11) and the final keyword in method overriding. Why is using override considered a standard best practice?

Question 10: Write a complete C++ program that demonstrates **Runtime Polymorphism**. Define a base class Parent with a virtual function show(), a derived class Child that overrides show(), and invoke show() in main() via a base class pointer (Parent* ptr).

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Does method overriding occur in the same scope or across an inheritance hierarchy?

Question 12 (Viva): What is the memory footprint overhead added to an object instance when its class includes or inherits virtual functions?

Question 13 (Viva): What happens if a base class destructor is non-virtual and a derived object created via new Child() is deleted using a base pointer (Parent* ptr)?

Question 14 (Viva): Is the virtual keyword required in a derived class when overriding a method that was already declared virtual in the base class?

Question 15 (Viva): What keyword introduced in C++11 forces the compiler to explicitly verify that a derived class method matches a base class virtual function signature?

Question 16 (Viva): Which structure stores the memory addresses of virtual functions for a class: the vptr or the vtable?

Question 17 (Viva): Where inside an object instance in RAM is the hidden vptr stored?

Question 18 (Viva): What keyword prevents a derived class from further overriding a virtual function down the inheritance tree?

Question 19 (Viva): What form of binding is used by default in C++ when invoking member functions without the virtual keyword?

Question 20 (Viva): Does object slicing occur when passing objects by pointer/reference or when assigning derived objects by value?

Answers & Explanations

Answer: 01

Method Overriding: Occurs when a derived class redefines a member function inherited from a base class using the exact same function signature (name, parameter list, and return type).

Function Overloading vs. Method Overriding:

Feature	Function Overloading	Method Overriding
Scope	Same class or global scope	Across Base and Derived classes (Inheritance)

Feature	Function Overloading	Method Overriding
Function Signature	Must differ in parameters (count or types)	Must be identical (name, parameters, return type)
Binding Type	Static / Compile-Time Binding	Dynamic / Runtime Binding (with virtual)
Inheritance	Not required	Mandatory inheritance relationship

Answer: 02

Identical Signature Requirement: Method overriding provides specialized derived behavior while maintaining a uniform base interface. The signature (name, parameters, constness, and return type) must match exactly so the compiler recognizes the method as an override.

Signature Mismatch / Function Hiding: If parameters differ slightly (e.g., void show(int x) in child vs. void show() in parent), the child method **hides** the base method instead of overriding it, disabling dynamic dispatch.

Answer: 03

Static Binding Gotcha: When a non-virtual function is overridden in a child class, function call resolution relies on static binding (determined at compile time based strictly on the pointer variable type).

Impact: Assigning a Child object to a Parent* pointer and calling ptr->show() executes Parent::show() instead of Child::show() because the compiler resolves the call based on the pointer type (Parent*), ignoring the actual target object type.

Answer: 04

Universal Remote Analogy: Without base pointers, a system requires separate remote controls for every single device (TV remote, AC remote, Fan remote), managing distinct object types independently. With base pointers, a single "Universal Remote Control" (Parent*) with generic buttons (virtual methods) controls any targeted appliance dynamically.

Runtime Flexibility: Base class pointers provide a single uniform interface to manipulate any derived object at runtime without needing to know its concrete class in advance or rewriting client logic when new subclasses are added.

Answer: 05

Virtual Function: A member function declared inside a base class using the virtual keyword, designed to be overridden in derived child classes.

Shift to Dynamic Binding: Marking a base function virtual instructs the compiler to postpone call resolution from compile time to runtime (Dynamic / Late Binding). Instead of calling a function based on the pointer's type, C++ inspects the actual object in memory at runtime via pointer lookup to invoke the correct overridden implementation.

Answer: 06

vtable (Virtual Table): A static array of function pointers generated by the compiler for every class that contains or inherits virtual functions. Each entry holds the physical memory address of the corresponding virtual function implementation.

vptr (Virtual Pointer): A hidden pointer added as the first member variable inside every object instance of a class with virtual functions. It points directly to the static vtable of that specific class.

Dynamic Dispatch: When invoking a virtual method, C++ dereferences the object's vptr, looks up the function address in its vtable, and performs an indirect call to execute the derived method.

Answer: 07

Step 1 (Dereference Pointer): The program dereferences ptr to access the target Child object memory block.

Step 2 (Fetch vptr): It reads the hidden vptr stored at the beginning of the object instance to locate the Child class vtable.

Step 3 (Table Index Lookup): It indexes into the Child vtable to retrieve the function address corresponding to show().

Step 4 (Indirect Jump): It jumps to the retrieved memory address (e.g., 0x004012A0) and executes the Child::show() code block.

Answer: 08

Definition: Object Slicing occurs when a derived class object is assigned directly to a base class value object (e.g., Parent p = childObj;).

Consequence: The base object p only has memory allocated for base data members. Assigning a child object copies over only the base sub-object attributes, stripping away all derived-level attributes and the child vptr. As a result, dynamic polymorphism is lost, and calling p.show() strictly executes Parent::show().

Answer: 09

override Specifier: Explicitly tags derived class functions to instruct the compiler to verify that a matching base virtual function signature exists. If signatures mismatch (e.g., due to typos or parameter differences), the compiler raises an explicit error instead of silently hiding the function.

final Keyword: Tags functions or classes to prevent further overriding or inheritance down the class hierarchy.

Answer: 10

```
#include <iostream>
using namespace std;
class Parent
{
  public:
    // Virtual function enables dynamic binding
    virtual void show()
    {
       cout << "Hello from Parent Class" << endl;
    }
    virtual ~Parent() = default; // Best practice virtual destructor
};
class Child : public Parent {public:
    // Overriding base virtual function
    void show() override
    {
       cout << "Hello from Child Class" << endl;
    }
};
int main()
{
    Child childObj;
    Parent* ptr = &childObj; // Base class pointer pointing to child object

    // Dynamic binding executes Child::show() at runtime
    ptr->show();

    return 0;
}
```

Answer: 11

Across an **inheritance hierarchy** (between Base and Derived classes).

Answer: 12

The physical size of **one pointer** (e.g., 8 bytes on a 64-bit architecture) to store the hidden vptr.

Answer: 13
Only Parent::~Parent() executes, causing potential **resource memory leaks** by skipping the Child destructor.

Answer: 14
No; once declared virtual in a base class, the function remains virtual across all derived classes automatically.

Answer: 15
The **override** specifier.

Answer: 16
The **vtable** (Virtual Table).

Answer: 17
As the **first hidden member variable** inside the object instance.

Answer: 18
The **final** keyword.

Answer: 19
Static Binding (Early Binding / Compile-Time Binding).

Answer: 20
When assigning derived objects **by value** to a base class object.